

Anchorline Project Partners — Final Project Handoff

Status key used throughout this document:

- **IMPLEMENTED** — built, working, verified in this codebase.
- **PARTIALLY IMPLEMENTED** — built in code but not fully wired end-to-end.
- **NOT IMPLEMENTED** — does not exist in this codebase.
- **REQUIRES CLIENT INPUT** — needs real content/decisions from the business owner.
- **REQUIRES EXTERNAL SERVICE CONFIGURATION** — code is written, but a real

third-party account/credential/domain must be supplied before it functions.

This document was produced by reading the actual repository on disk — file listings, `package.json`, `.env.example`, and `git log` were all checked directly rather than recalled from memory. Nothing below is aspirational; every "IMPLEMENTED" claim corresponds to a file that exists in this repository as of the commit noted in Section AC.

A. Project Overview

- **Project name:** Anchorline Project Partners — Official Business Website
- **Business:** Anchorline Project Partners, an independent construction

oversight, quantity surveying, and project management consultancy led by Damian Chibueze Agu, based in Abuja, Nigeria.

- **Purpose:** build credibility with diaspora and private clients who

cannot personally supervise their construction projects, explain the three-tier service model, and convert visitors into WhatsApp/form enquiries — with a client-operated content dashboard so the business can update the site without needing a developer for routine text changes.

- **Source of truth for business content:**

`docs/client-brief/Anchorline_Detailed_Website_Brief.docx`. Business facts, service names, and claims are drawn from this brief; nothing about the business itself has been invented.

- **Design direction — recorded, approved change:** the brief specifies

a "clean, minimal, no heavy animation" site. The user explicitly authorized overriding that in favor of a cinematic, interactive experience (scroll-driven construction sequence, staged entrances, hover interactions). This is a deliberate, on-record departure from the brief's original design instruction, not an unauthorized change.

- **Status: IMPLEMENTED** (codebase). **NOT DEPLOYED**. **NOT yet reviewed by the client.**

B. Original Client Requirements

From the client brief, the core requirements were:

- 1 A credible, professional website for an independent construction oversight/QS/PM consultancy.

- 1 Explain three locked service tiers: **WATCH, VERIFY, MANAGE**.

- 2 Tagline: *"Your eyes on the ground, wherever you are."*

- 3 Primary contact channels: WhatsApp (**0806 757 0941**) and email

(**alprojectpartners@gmail.com**).

- 1 Convert visitors (especially the Nigerian diaspora) into enquiries via a contact form and WhatsApp.

- 1 "Clean, minimal, no heavy animation" visual direction (**later explicitly overridden by the client/user in favor of a cinematic, interactive direction — see Section A**).

- 1 No fabricated content: no invented client names, project addresses, costs, statistics, or completion dates anywhere on the site.

All of the above are **IMPLEMENTED**, with the design-direction item carrying the recorded override noted above.

C. Implemented Features

IMPLEMENTED:

- Full public marketing site (10 routes — see Sitemap, Section D).
- Three-tier service model (WATCH / VERIFY / MANAGE) presented on the homepage and in full detail on /services.

- Scroll-driven, code-generated CSS 3D construction visualization

(Section J).

- Real, working client content-management system: login → dashboard →

edit → save draft → preview → publish → live (Section L).

- Image and PDF upload for media and the sample report (through the

dashboard).

- Contact form with server-side validation, honeypot spam trap, and

rate limiting (Section N).

- WhatsApp click-to-chat integration, sourced from dashboard-editable

contact info (Section O).

- SEO file-convention routes (`sitemap.xml`, `robots.txt`), per-page

metadata, and JSON-LD structured data on the homepage (Section P).

- Official Anchorline logo, favicon, and social-share (Open Graph)

image, all integrated from client-supplied source files (Section I).

- Responsive layout verified at six breakpoints (Section T).
- `prefers-reduced-motion` fallback for every animated section.

PARTIALLY IMPLEMENTED:

- SEO per-page title/description overrides — the dashboard UI and

database model exist, but public pages have **not yet been wired**

to read the override; they still render their original hardcoded

metadata (see Section P and Section AB).

NOT IMPLEMENTED:

- Insights/blog page (explicitly optional / Phase 2 in the brief).
- Basic analytics (no provider chosen or wired in).
- Admin user role distinct from the single "client" role (the auth

system's session payload has a `role` field ready for this, but no

second role or permission tier exists today).

D. Complete Sitemap

Route	Purpose	Auth
/	Home — hero, construction visualization, trust strip, tier snapshot, How It Works teaser, closing CTA	Public
/about	Principal Consultant bio, credentials, approach	Public
/services	Full WATCH / VERIFY / MANAGE tier detail	Public
/how-it-works	5-step client journey, scroll-activated	Public
/sample-report	Report structure preview + status	Public
/coverage-area	Abuja-based coverage, interstate arrangements	Public
/faq	FAQ accordion	Public
/testimonials	Client testimonials or honest "collecting feedback" placeholder	Public
/contact	Get-a-Quote form + WhatsApp/email panel	Public
/design-system	Internal brand/component reference (noindex)	Public, not indexed
/login	Client dashboard login	Public form, protected destination
/dashboard and /dashboard/*	Client CMS — Overview, About, Services, How It Works, FAQ, Testimonials, Sample Report, Media, Contact Information, SEO, and a /dashboard/preview/[section] draft-preview route	Protected — requires login
/sitemap.xml	Auto-generated XML sitemap	Public
/robots.txt	Auto-generated robots file	Public

E. Technical Architecture

- **Framework:** Next.js 16 (App Router), React 19, TypeScript.
- **Rendering:** the entire site is dynamically rendered

(export const dynamic = "force-dynamic" in the root layout),
because content can change via the dashboard and must reflect immediately without a rebuild. This is a deliberate tradeoff from a previously fully-static site.

- **Route groups:** src/app/(site)/ groups all public pages under a

layout that renders the public Header/Footer. src/app/dashboard/ and src/app/login/ sit outside that group with their own chrome —

the dashboard never shows the public nav, and the public site never shows dashboard chrome. ((site) is invisible in URLs.)

- **Data layer:** Node's built-in `node:sqlite` module — no external

database dependency. One table, `content_sections`, keyed by section name, storing separate `draft_json` and `published_json` columns.

- **Auth:** Node's built-in `crypto` module — script password hashing,

HMAC-SHA256 signed session cookies. No third-party auth library.

- **File uploads:** written directly to `public/uploads/` on the local

filesystem, with server-generated random filenames.

F. Technology Stack and Versions

Read directly from `package.json` and the runtime environment:

Package	Version
next	16.3.0
react / react-dom	19.2.8
typescript	^5 (installed: 5.9.3)
tailwindcss	^4
eslint / eslint-config-next	^9 / 16.3.0
Node.js (build/runtime)	v22.22.2 (required — uses <code>node:sqlite</code> , experimental as of Node 22)
Package manager	npm, <code>package-lock.json</code> committed

****Zero new npm dependencies were added for the CMS or the construction visualization.**** Storage, auth, and the 3D scene all use Node/browser built-ins (`node:sqlite`, `crypto`, CSS 3D transforms) rather than adding a database driver, auth library, or 3D framework (Three.js/WebGL were deliberately not used).

G. Design System / Brand System

- **Brand colors, typography, spacing, and component tokens** are

defined in `src/app/globals.css` as Tailwind v4 design tokens

(`--color-navy-*`, `--color-gold-*`, etc.).

- **Locked service tier names** (WATCH / VERIFY / MANAGE), the

tagline, navigation structure, and the sample-report's fixed section labels live in `src/content/site.ts` as code constants — **never** exposed as editable in the dashboard, since these are locked brand facts, not day-to-day content.

- **Internal reference page:** `/design-system` renders every color,

type scale, and shared UI component live, for developer/designer reference. It is noindexed and not part of the public sitemap.

H. Homepage

`src/app/(site)/page.tsx`. Sections, top to bottom: hero (staged entrance animation), scroll-driven construction visualization (Section J), trust-pillar strip, service-tier snapshot cards, "How It Works" teaser with scroll progression, closing CTA banner with WhatsApp + Get-a-Quote buttons. JSON-LD structured data is generated per-request (not at module load) so it always reflects the current dashboard-configured contact info.

I. Official Anchorline Logo Integration

- **Logo file:** `public/assets/logo/anchorline-logo.png` — the

client-supplied official logo, used as-is (no redesign), rendered in the site header via `src/components/layout/Logo` inside `src/components/layout/Header.tsx`.

- **Favicon:** `src/app/icon.png` (512×512), `src/app/apple-icon.png`

(180×180), `src/app/favicon.ico` (multi-size 16/32/48) — all extracted from the client-supplied favicon design sheet (`public/assets/brand/anchorline-favicon-design-original.png`, preserved unmodified) using the clean square icon variant on that sheet. Next.js's file-convention system auto-generates the correct `<link rel="icon">`/`<link rel="apple-touch-icon">` tags — verified these URLs return HTTP 200 and the tab icon renders.

- **Social-share (Open Graph) image:**

`public/assets/brand/og-image.png` (1200×630) — extracted from the client-supplied social-share design sheet

(public/assets/brand/anchorline-social-share-design-original.png, preserved unmodified), with the sheet's own "SOCIAL SHARE / OPEN GRAPH IMAGE" mockup watermark cropped out. Wired into `openGraph.images/twitter.images` in `src/app/layout.tsx`. **Note:** the supplied graphic contains its own baked-in marketing copy ("Protecting your investment. From the ground up.") that differs slightly from the brief's approved tagline — used as-supplied since it's a designed creative asset for social previews only, not on-page content.

Status: IMPLEMENTED.

J. Construction Visualization

Status: IMPLEMENTED.

- **File:** `src/components/motion/House3D.tsx`, used by

`src/components/sections/BuildingStory.tsx`.

- **CSS 3D architecture:** pure CSS 3D transforms

(`perspective + transform-style: preserve-3d + translate3d/rotateX/rotateY`) — ****no Three.js, WebGL, or canvas, and no images.**** A `Face` primitive renders one 3D-positioned plane; a `Block` primitive composes the front, right, and top faces of a box (the only faces the fixed camera angle ever sees) into foundation, structural columns, walls, and roof volumes. Materials (concrete, painted render, slate roofing, glass) are CSS gradients.

- **Construction stages** (9, matching progressive scroll position):

Foundation → On Site (structural columns) → Structure/Walls → Roof → Windows/Doors/Exterior finishes → Interior (lit windows) → Inspect → Verify → Report.

- **Scroll-driven progression:** a 620vh-tall section with a sticky

inner viewport; a scroll-progress hook

(`src/components/motion/useScrollProgress.ts`) computes a 0–1 value from normal page scroll (rAF-throttled) that drives every stage's opacity/geometry. This is **not scroll-jacking** — the browser's native scroll behavior is untouched; the visualization simply reacts

to scroll position.

- **Cinematic camera movement:** a slow, restrained camera pan

(rotateY interpolating across the whole sequence) gives a three-quarter cinematic view without any scroll-hijacking or auto-playing video.

- **Realistic architectural styling:** proportioned single-family

house massing, a pitched roof with a gold brand-accent trim line, window/door openings on both visible wall faces, a driveway wedge, and simple landscaping — deliberately restrained/geometric rather than photorealistic, appropriate for a code-generated illustration.

- **Illustrative disclosure:** a visible caption directly beneath the

visualization reads `'''Illustrative architectural visualization — not a photo of an actual Anchorline project.'''` The scene container also carries `role="img"` and a matching `aria-label` for assistive technology. No real client name, address, cost, or completion date is implied anywhere.

- **Mobile behaviour:** verified via Playwright screenshots at

375/390/414px — the visualization scales into the available column width, no horizontal overflow, no console errors.

- **Reduced-motion behaviour:** when `prefers-reduced-motion: reduce`

is set, the entire pinned 3D sequence is replaced with a static, fully-visible 9-card text grid describing every stage — no information depends on the animation, and no 3D scene renders at all in this mode.

- **Two rendering bugs found and fixed during QA** (documented for the

next developer, not swept under the rug):

1 An early version placed the ground/driveway as literal 3D planes underneath the house; the browser's 3D depth-sort proved unreliable for a large plane sharing screen space with much smaller building geometry (CSS 3D has no true per-pixel z-buffer). Fixed by rendering grounding as a flat 2D layer behind the 3D scene instead of inside it.

1 Window/door graphics were originally a separate face stacked exactly on top of the wall face; the browser's compositor sorted that pairing inconsistently depending on scroll position (confirmed via `elementFromPoint()` — hit-testing was correct, paint order was not). Fixed by nesting the window/door elements as direct children of the wall's own face element, removing the ambiguity entirely.

K. Navigation and Responsive Behaviour

- **Header:** `src/components/layout/Header.tsx` — desktop horizontal nav plus WhatsApp/Get-a-Quote buttons; a mobile hamburger menu at narrower widths.
- **Footer:** `src/components/layout/Footer.tsx` — repeats primary nav, contact info, and social/WhatsApp links.
- **Responsive verification:** tested at 375, 390, 414 (mobile), 768 (tablet), and 1280, 1440 (desktop) px — 0 horizontal overflow, 0 console errors, 0 broken images across all public routes plus `/login` (Section T has the full QA record).

L. Client Management System — Status: IMPLEMENTED

This is a genuine, working system — not a mockup or a stub. Verified files on disk:

- `src/lib/auth.ts` — password hashing (scrypt) + session token creation/verification (HMAC-SHA256).
- `src/lib/dashboard-auth.ts` — server-side session guard used by every protected API route.
- `src/app/login/page.tsx` + `LoginForm.tsx` — the login page and form.
- `src/app/api/auth/login/route.ts`, `src/app/api/auth/logout/route.ts` — session cookie issuance/clearing.
- `src/app/dashboard/layout.tsx` — protected shell; redirects to `/login` if no valid session.

- ``src/app/dashboard/{about,services,how-it-works,faq,testimonials, sample-report,media,contact,seo}/page.tsx`` — one editor per content section, plus `src/app/dashboard/page.tsx` (Overview).
- `src/app/dashboard/preview/[section]/page.tsx` — protected preview of **draft** (unpublished) content through the real page components.
- `src/app/api/dashboard/content/[section]/route.ts` (GET draft / PUT save draft) and `.../publish/route.ts` (POST — publish).
- `src/app/api/dashboard/upload/route.ts` — image/PDF upload.

Flow, exactly as implemented:

LOGIN → DASHBOARD → EDIT → SAVE DRAFT → PREVIEW → PUBLISH → LIVE

1 Client logs in at `/login` with credentials from environment variables (see Section U).

1 Dashboard sidebar: Overview, About, Services, How It Works, FAQ, Testimonials, Sample Report, Images/Media, Contact Information, SEO.

1 Editing a section writes only to that section's **draft** row in the database — nothing on the live site changes yet.

1 **Preview** opens `/dashboard/preview/<section>` and renders the draft through the real, live page components, behind a visible "PREVIEW — not published" banner.

1 **Publish** copies the draft into the **published** row — the change is live immediately, no rebuild or redeploy required (because rendering is dynamic — see Section E).

Verified end-to-end in this project (not just code-reviewed): the About section was edited, saved as a draft, confirmed to show an "Unpublished changes" indicator, published, and the change was confirmed live on `/about` via a direct HTTP request.

What is NOT yet configured (see Section U/Y): no real `CLIENT_USERNAME/CLIENT_PASSWORD_HASH/SESSION_SECRET` values exist in any environment — the dashboard is fully built but cannot actually be logged into until an operator generates and sets these three values.

M. Content Architecture

- **Storage:** `node:sqlite`, table `content_sections` — one row per

content section, columns `draft_json`, `published_json`,
`updated_at`, `published_at`.

- **Seeding:** on first run, `src/lib/seed.ts` migrates each section's previous static content `/*.json` file into the database (About, Services, How It Works, Coverage Area, FAQ, Testimonials), and seeds three new database-only sections — `contact` (WhatsApp/email), `seo` (per-page overrides, starts empty), `sample-report` (uploaded file path/note, starts empty) — from the locked constants they used to come from.

- **Client-editable via the dashboard:** About narrative/approach/credentials, Services tier *descriptions* (not names), How It Works step copy, FAQ items, Testimonials, Contact info (WhatsApp number, email), Sample Report file, uploaded media, per-page SEO title/description (not yet read by the pages — see Section AB).

- **Locked in code, never editable via the dashboard**

(`src/content/site.ts`): business name, tagline, site navigation, service tier **names** (WATCH/VERIFY/MANAGE), sample-report's fixed section labels.

- **No runtime schema validation** on content saves — the API accepts any JSON matching the section's TypeScript shape by convention, not by an enforced schema. Since only the single authenticated client role can reach these endpoints, the practical risk is limited to the client accidentally saving malformed content for their own site, not a cross-user vulnerability. Documented as a known limitation (Section AB), not a security hole.

N. Contact Form and Email Integration

- **Endpoint:** `POST /api/contact` (`src/app/api/contact/route.ts`).
- **Fields:** Name, Email/Phone, Project Location, Project Stage, Tier

of Interest, Message.

- **Validation:** server-side only, honeypot hidden field for basic

bot filtering, per-IP rate limiting (5 submissions / 10 minutes).

- **Destination email:** sourced from the dashboard's Contact

Information section by default; `CONTACT_TO_EMAIL` env var available as a testing-only override.

- **Email delivery:** implemented in code to call Resend

(`https://resend.com`) via `fetch()` — ****REQUIRES EXTERNAL SERVICE CONFIGURATION.**** No `RESEND_API_KEY` or `CONTACT_FROM_EMAIL` is set anywhere in this repository or any environment. Until they are, the form validates and rate-limits correctly but returns an honest HTTP 503 rather than pretending to send mail.

****In the requested phrasing: "Implemented in code — not connected to a live Resend account."****

O. WhatsApp Integration

Status: IMPLEMENTED. The number and `wa.me` link are sourced from the dashboard-editable Contact Information section

(`getContactInfo()` + `whatsappHrefFor()` in `src/lib/content.ts`), not a hardcoded constant. Appears in: header (desktop + mobile), footer, every page's closing CTA banner, and the Contact page's dedicated panel. The link is derived by stripping non-digit characters from whatever number the client enters — the dashboard displays a warning that the country code must be included with no spaces so the click-to-chat link keeps working.

P. SEO / Metadata / Sitemap / Robots

- `src/app/sitemap.ts` — **IMPLEMENTED**, generates `/sitemap.xml` from

the live navigation list.

- `src/app/robots.ts` — **IMPLEMENTED**, generates `/robots.txt`.
- Per-page `<title>/meta` description — **IMPLEMENTED** as hardcoded

defaults per page.

- JSON-LD structured data on the homepage — **IMPLEMENTED**, computed

per-request so it reflects live dashboard contact info.

- Dashboard SEO override section (custom title/description per page) —

PARTIALLY IMPLEMENTED: the data model, database section, and dashboard editing UI all exist and save correctly, but no public page's `generateMetadata()` has been wired to read these overrides yet. Pages currently render their original hardcoded metadata regardless of what's saved in the SEO dashboard section. This is an honest, documented gap (see Section AB), not something silently left looking finished.

- `NEXT_PUBLIC_SITE_URL` — **REQUIRES EXTERNAL SERVICE CONFIGURATION**

(i.e., a chosen production domain). Not set anywhere; sitemap/robots/canonical/OG URLs currently fall back to `http://localhost:3000`.

Q. Accessibility

Status: IMPLEMENTED (verified, not just coded):

- Semantic HTML landmarks, one `<h1>` per page (verified across all

QA'd routes/breakpoints).

- Keyboard navigation and visible focus states on all interactive

elements, including dashboard forms and the login form.

- Honeypot spam field hidden accessibly (not just visually).
- `role="alert"` on form error states.
- Reduced-motion respected everywhere animation is used (hero,

construction visualization, How It Works scroller, scroll-reveal utilities) — see Section J for the construction visualization's specific fallback.

R. Performance

- **Zero new npm dependencies** added for either the CMS or the

construction visualization — `node:sqlite`, `crypto`, and CSS 3D transforms are all Node/browser built-ins.

- The construction visualization is driven entirely by CSS transform and opacity, both compositor-only properties — no layout/paint thrashing, cheap on mobile despite the visual depth.
- Additional client-side JavaScript from this project's CMS work (forms, upload widgets, login/logout) is scoped to /login and /dashboard only — never loaded on public pages.
- **Tradeoff, made deliberately:** the site moved from fully static generation to dynamic rendering (force-dynamic) so that publishing a dashboard edit takes effect immediately without a rebuild. Pages remain lightweight (a small number of SQLite reads per request, no heavy per-request computation).
- No formal Lighthouse/Core Web Vitals audit has been run in this project — **NOT IMPLEMENTED / NOT MEASURED.**

S. Security

Status: IMPLEMENTED, verified by direct code inspection:

- **Passwords:** script-hashed with a salt (Node's built-in crypto), never stored or logged in plaintext.
- **Sessions:** HMAC-SHA256 signed cookies, httpOnly (inaccessible to page JavaScript/XSS), sameSite: "lax" (mitigates CSRF on state-changing requests), secure flag set in production, 12-hour expiry.
- **Timing-safe comparisons:** username and password checks use crypto.timingSafeEqual.
- **Authorization defense-in-depth:** every dashboard page *and* every /api/dashboard/* route independently verifies the session server-side — the layout-level check alone would not be sufficient against direct API calls, so each route checks for itself too.
- **Brute-force protection:** the login endpoint is rate-limited (5 attempts / 10 minutes / IP), same mechanism as the contact form.
- **File uploads:** MIME-type allowlist (image/png, image/jpeg,

image/webp, plus application/pdf for reports), 10MB size cap, server-generated random filenames — a client-supplied filename never touches the filesystem path.

- **Secrets:** read only inside server-side modules, never referenced

from any client component or present in a browser bundle; all

.env* files are gitignored except .env.example (which contains only empty placeholders — verified, see Section U).

- **Known, accepted limitation:** content-mutation API routes validate

JSON parseability and a 200KB size cap, but do not validate the internal shape of each section against a strict runtime schema (see Section M). Given the single trusted client role, this is an accepted, documented risk rather than an oversight.

T. Testing and QA

Performed and verified in this project:

- `npm run lint` — clean (0 errors/warnings).
- `npm run build` — clean (0 TypeScript errors), all routes compile

including dashboard/API routes.

- Live browser QA (Playwright/Chromium) across 375, 390, 414, 768,

1280, 1440px on all 10 public routes plus `/login`: 0 console

errors, 0 horizontal overflow, 0 broken images, exactly one `<h1>` per page.

- **Login flow:** unauthenticated `/dashboard` access redirects to

`/login` (verified); valid login redirects to `/dashboard`

(verified); logout clears the session (verified).

- **Content editing, end-to-end:** edited the About draft, saved,

confirmed the "Unpublished changes" indicator, published, confirmed

"Published — live now," then confirmed via a direct HTTP request

that the edit appeared on the live `/about` page.

- **Route-group regression caught and fixed:** an earlier build had

the dashboard rendering inside the public site's Header/Footer

(duplicate customer-facing nav above the CMS). Restructured into a

(site) route group and re-verified both the dashboard (clean) and public site (unaffected) afterward.

- **Favicon/OG:** verified all icon/OG URLs return HTTP 200 and the correct `<link>/<meta>` tags are present in rendered HTML.

- **Construction visualization:** Playwright screenshots at 8 scroll checkpoints across 1440px and 390px, plus a ``prefers-reduced-motion: reduce`` pass — 0 console errors, correct 3D geometry at every checkpoint (no mirrored/inside-out faces, no z-fighting after the two fixes described in Section J), all stage elements present and stable across the full scroll range.

- Mobile dashboard screenshot confirms a usable layout at 375px.

Not performed: automated unit/integration test suite (none exists in this repository — testing has been manual + Playwright screenshot QA throughout), formal accessibility audit tooling (axe, Lighthouse), load/performance testing, cross-browser testing beyond Chromium.

U. Environment Variables

Every variable the application reads, confirmed against

`.env.example` (verified: every value in that file is an empty placeholder — no secrets are committed):

Variable	Purpose	Status
NEXT_PUBLIC_SITE_URL	Production domain for sitemap/robots/OG/canonical URLs	Not set — falls back to <code>http://localhost:3000</code>
RESEND_API_KEY	Contact form email delivery	Not set — no Resend account exists
CONTACT_FROM_EMAIL	Verified Resend sender address	Not set
CONTACT_TO_EMAIL	Optional test override of destination inbox	Not set (defaults to the dashboard-configured contact email)
CLIENT_USERNAME	Dashboard login username	Not set — dashboard cannot be logged into until this is set
CLIENT_PASSWORD_HASH	Dashboard login password, <code>scrypt</code> -hashed	Not set — generate with <code>node scripts/hash-password.mjs "the real password"</code>
SESSION_SECRET	Signs dashboard session cookies	Not set — generate with <code>openssl rand -hex 32</code>

****None of these have real values anywhere in this repository or in any committed file.****

V. Assets and Asset Locations

Asset	Location
Official logo	public/assets/logo/anchorline-logo.png
Favicon (512×512)	src/app/icon.png
Apple touch icon (180×180)	src/app/apple-icon.png
Favicon.ico (multi-size)	src/app/favicon.ico
Original favicon design sheet (unmodified)	public/assets/brand/anchorline-favicon-design-original.png
Extracted master icon	public/assets/brand/anchorline-icon-master.png
Social-share / OG image (1200×630)	public/assets/brand/og-image.png
Original social-share design sheet (unmodified)	public/assets/brand/anchorline-social-share-design-original.png
Construction visualization component	src/components/motion/House3D.tsx
Construction visualization wrapper/copy	src/components/sections/BuildingStory.tsx
Auth/session logic	src/lib/auth.ts
Dashboard API auth guard	src/lib/dashboard-auth.ts
Content database layer	src/lib/db.ts
Content read/write helpers	src/lib/content.ts
First-run content seeding	src/lib/seed.ts
Dashboard pages	src/app/dashboard/
Login page	src/app/login/
Contact form API	src/app/api/contact/route.ts
Environment variable template	.env.example
Client brief (source of truth)	docs/client-brief/Anchorline_Detailed_Website_Brief.docx
Locked business constants	src/content/site.ts
Password-hash generator script	scripts/hash-password.mjs

W. What Is Connected to Real External Services

Nothing. As of this handoff, no external third-party service has live credentials configured anywhere in this repository or any known

environment:

- No Resend account/API key.
- No production domain.
- No configured dashboard login credentials.
- No analytics provider.
- No object storage provider (not needed for the current

local-filesystem-based upload approach — see Section Y).

- Not deployed to any hosting provider.

X. What Is NOT Connected to Real External Services

Everything in Section W. To restate plainly, using the requested phrasing style:

- **Contact form email:** "Implemented in code — not connected to a

live Resend account."

- **Client dashboard login:** "Implemented in code — not connected to

any real, configured username/password/session secret."

- **Production domain:** "Not configured — the site currently only

resolves at localhost."

- **Deployment:** "Not deployed."
- **Analytics:** "Not implemented — no provider chosen."

Y. Remaining Client Requirements

Items that need the actual business owner's input, decisions, or files before this project can go fully live:

1 Dashboard credentials — the operator needs to choose a username/password and generate the corresponding CLIENT_USERNAME/CLIENT_PASSWORD_HASH/SESSION_SECRET values (Section U). No credentials exist yet — this must happen before the client can log in at all.

1 Resend account — sign up, verify a sending domain, obtain RESEND_API_KEY, decide on CONTACT_FROM_EMAIL.

1 **Production domain** — choose and configure, then set

NEXT_PUBLIC_SITE_URL.

1 **Final approved copy** for About/Services/How It Works — the

current text is the brief's own draft language, editable via the dashboard once real copy is approved.

1 **Anonymised sample report file** — to be uploaded via the

dashboard's Sample Report page once ready; the site currently shows a clearly labeled placeholder, never fabricated content.

1 **Real project photography**, if the client later wants to replace

or supplement the illustrative CSS 3D visualization with actual project photos — not required, but noted as an option.

1 **Real client testimonials**, added via the dashboard only with the

testimonial-giver's permission — the site shows an honest "collecting feedback" message until any exist.

1 **Decision on hosting** — a persistent-filesystem host (current

architecture, e.g. a traditional Node server or a platform with persistent disk) vs. a serverless/edge host (would require swapping node:sqlite for a hosted database and local file uploads for object storage — not a small change, a decision to make deliberately before committing to a host).

Z. Local Development Instructions

```
# 1. Install dependencies (Node 22.x required)
npm install

# 2. Generate dashboard credentials
openssl rand -hex 32 # → SESSION_SECRET
node scripts/hash-password.mjs "choose a password" # → CLIENT_PASSWORD_HASH

# 3. Create your local environment file
cp .env.example .env.local
# then fill in CLIENT_USERNAME, CLIENT_PASSWORD_HASH, SESSION_SECRET
# (RESEND_API_KEY / CONTACT_FROM_EMAIL / NEXT_PUBLIC_SITE_URL are optional locally)

# 4. Run the dev server
npm run dev
```

- Public site: <http://localhost:3000>
- Client login: <http://localhost:3000/login>

- Internal brand/component reference: `http://localhost:3000/design-system`

Without the three dashboard env vars set, the ****public** site works

fine** — only `/login` and `/dashboard` will reject every login

attempt with a clear "not configured" error (not a silent failure).

AA. Production Deployment Instructions

This project has not been deployed by any session working on it.

The following is guidance for whoever performs the first deployment,

not a record of a deployment that has happened:

- 1 **Choose a host with a persistent, writable filesystem.** The CMS

database (`data/anchorline.db`) and uploaded files

(`public/uploads/`) both live on local disk. A serverless/edge host

(e.g. platforms that run functions statelessly) will **not** work

as-is — a hosted database and object storage would need to replace

`node:sqlite` and local file writes first.

- 1 Set all required environment variables on the host (Section U) —

real, unique `CLIENT_PASSWORD_HASH/SESSION_SECRET` values, a real

`NEXT_PUBLIC_SITE_URL`, and Resend credentials if email delivery is

wanted at launch.

- 1 `npm run build` && `npm run start` (or the host's equivalent build/

start commands).

- 1 Verify HTTPS is enforced (the session cookie's `secure` flag

depends on `NODE_ENV=production`, which most hosts set

automatically).

- 1 Confirm the dashboard login works with the real production

credentials before handing the site to the client.

- 1 Push the repository to GitHub only when explicitly instructed —

all current work is committed locally (Section AC).

AB. Known Limitations

- **Dashboard cannot be used yet** — no login credentials are

configured in any real environment (by design; none were invented on the project's behalf).

- **Email provider not connected** — contact form validates and rate

limits correctly but cannot send real mail until a Resend account is configured.

- **Production domain not configured.**
- **SEO dashboard section not wired into page metadata yet** — the

data model and dashboard UI exist and save correctly, but public pages still render their original hardcoded titles/descriptions rather than the saved override (Section P).

- **No runtime schema validation on content saves** (Section M/S) —

accepted, documented risk given the single trusted client role.

- ****node:sqlite + local file uploads require a persistent-filesystem**

host** — will not work as-is on a serverless/edge host without further changes (Section Y).

- **Client content still pending from the actual business owner:**

final approved About/Services/How-It-Works copy, an anonymised sample report, testimonials (Section Y).

- **No automated test suite** — verification throughout this project

has been manual QA plus Playwright screenshot-driven checks, not a unit/integration test suite.

- ****No formal performance or accessibility audit tooling has been**

run** (e.g. Lighthouse, axe) — accessibility and performance claims in Sections Q/R reflect manual verification, not automated scoring.

AC. Git Status

- **Branch:** `claude/anchorline-project-partners-rbz3iz`
- **Working tree:** clean at the time this document was written (no

uncommitted changes).

- **Most recent commits** (newest first, from `git log`):

```
d14d316 Update handoff doc and README for the CSS 3D construction visualization
2b5b502 Replace SVG building illustration with code-generated CSS 3D visualization
```

b7cb5d7 Consolidate handoff docs into a single ANCHORLINE-HANDOFF.md
00f9051 Client-managed CMS, favicon/OG integration, and extended building story
8ab570e Interactive experience upgrade (approved override of the brief's minimal-design requirement)
b8d9d2e Update handoff docs: git status, production status, next-operator handoff
7931627 Add official Anchorline logo
c0ad335 Phase 10: final production build verification and project handoff
c1be8fb Phase 7-9: content editing, contact functionality, and QA pass
0459b3d Phase 4-6: global foundation, homepage, and core pages

- **Push status:** work is committed **locally only**. Per explicit

instruction, this project does not push to GitHub unless the user directly instructs it.

AD. Final Project Status

CODEBASE:	COMPLETE, BUILDS CLEAN (lint + build both pass)
DESIGN:	COMPLETE (cinematic direction, user-approved override)
LOGO / FAVICON / OG:	INTEGRATED (client-supplied source files, verified live)
CLIENT CMS:	BUILT AND VERIFIED END-TO-END – credentials not yet configured
CONSTRUCTION VISUALIZATION:	CSS 3D, 9 stages, code-generated (no photography, no Three.js/WebGL)
CONTACT FORM:	BUILT, VALIDATED – email delivery not connected (no Resend account)
WHATSAPP:	WORKING, dashboard-editable
SEO FILES (sitemap/robots):	WORKING – per-page SEO override UI built but not yet wired to output
ACCESSIBILITY:	MANUALLY VERIFIED – no automated audit run
SECURITY:	IMPLEMENTED (hashing, signed sessions, rate limiting, upload validation)
QA:	PASSED (manual + Playwright screenshot QA – no automated test suite)
PRODUCTION BUILD:	PASSED
DASHBOARD LOGIN:	NOT CONFIGURED (no credentials set anywhere)
DOMAIN:	NOT CONFIGURED
DEPLOYMENT:	NOT PERFORMED
GITHUB PUSH:	NOT PERFORMED (local commits only, by instruction)
CLIENT REVIEW:	PENDING

In one sentence: the codebase is complete, tested, and ready for a developer or the client to run locally today; nothing has been deployed, no external service has live credentials, and the dashboard cannot be logged into until real credentials are generated and set.